Guía para explicar el laboratorio de seguridad Umbral

Implementación demostraciones y correcciones

Trabajo Práctico Integrador de Seguridad en el Desarrollo de Software

Construimos una billetera ficticia en dos versiones comparables y una guía que permite demostrar seis vulnerabilidades de forma manual y automática. Este documento sirve para entender qué implementamos, mostrar el efecto de cada falla y explicar el control que incorporamos en la versión corregida.

La ejecución conservada como evidencia reprodujo las seis fallas en la versión vulnerable y verificó los seis controles en la corregida. El alcance es este laboratorio local y sus escenarios definidos; la expresión versión corregida no implica una garantía de seguridad absoluta.

Cómo utilizar este documento

Primero repasá la arquitectura y el procedimiento de reinicio. Después, cada ejercicio presenta el funcionamiento legítimo, la falla deliberada, los pasos para reproducirla, la corrección y el resultado efectivamente observado. Las frases de exposición son sugerencias para explicar el caso con tus propias palabras.

Contenido	Página
Aplicaciones y funcionamiento compartido	2
Preparación y comparación de versiones	3
Control de acceso administrativo	4
CSRF en transferencias	5
XSS persistente en conceptos	6
Inyección SQL en actividad	7
Enumeración e intentos de autenticación	8
SSRF en el cotizador	9
Automatización y resultados comprobados	10
Guion y preguntas para la exposición	11
Límites pendientes y archivos de referencia	12

Base documental: código del proyecto, guía existente y evidencias exportadas. Revisión del documento: 8 de noviembre de 2026. Las instrucciones manuales son un marco para el ensayo; los resultados numéricos identificados como observados proceden del registro automático conservado.

Aplicaciones y funcionamiento compartido

Las tres aplicaciones principales comparten una experiencia visual, pero cumplen funciones distintas. Una página auxiliar permite demostrar solicitudes entre orígenes sin depender de servicios externos.

Aplicación	Dirección local	Función
Vulnerable	http://localhost:5000	Billetera con seis fallas deliberadas.
Corregida	http://localhost:5001	Mismas operaciones con controles aplicados.
Guía	http://localhost:5002	Pasos manuales, ejecución y evidencias.
Página atacante CSRF	http://localhost:5003	Formulario local de demostración.

Qué implementamos en las dos billeteras

El frontend está construido con React, TypeScript, Vite y CSS propio. Flask sirve el mismo frontend compilado y la API mediante rutas relativas /api. Las pantallas permiten registrarse, iniciar y cerrar sesión, consultar el saldo, transferir, buscar movimientos, administrar ajustes y consultar una cotización ficticia. Cada versión se identifica visualmente.

Python y Flask implementan las operaciones; SQLite guarda usuarios, transferencias, ajustes y contadores de autenticación. El modo vulnerable o secure queda fijado al arrancar el proceso. Cambiar un parámetro del navegador no cambia el modo del servidor. Cada instancia usa su propia base, almacenamiento de sesiones y nombre de cookie.

Controles comunes que preservan el funcionamiento legítimo

Las contraseñas se almacenan como hashes mediante Werkzeug. Flask-Session y CacheLib guardan las sesiones en el servidor, regeneran su identificador al autenticar y permiten revocarlas al salir. La duración configurada es de 30 minutos. El reinicio invalida las sesiones mediante una generación de datos.

El dinero se guarda en centavos enteros: 100000 centavos representan $1.000,00 ficticios. Antes de transferir se valida el importe, el destinatario y el saldo. El débito, el crédito y el registro del movimiento forman una transacción SQLite; el débito condicional evita gastar saldo insuficiente ante solicitudes concurrentes. No se permite transferirse a uno mismo.

Cómo se contiene el laboratorio

Docker Compose conecta las billeteras, la guía y dos servicios simulados en una red interna. Una pasarela publica únicamente los cuatro puertos locales y reenvía a destinos fijos. El proveedor y el servicio interno no publican puertos. Incluso la demostración vulnerable de SSRF se limita a esos dos servicios ficticios.

El canal privado de reinicio está autenticado y separado de la API pública de la billetera. La guía no recibe comandos arbitrarios ni tiene acceso al socket de Docker. Todas las cuentas, los saldos y los datos internos del ejercicio son ficticios.

Preparación y comparación de versiones

Puesta en marcha y cuentas

Desde la carpeta tpi-billetera, con Docker disponible y las dependencias preparadas, iniciar el laboratorio y abrir la guía. Para un ensayo sin Internet, las imágenes deben estar construidas o descargadas de antemano.

```
docker compose up --build
```

Usuario	Contraseña de demostración	Rol	Saldo inicial
ana	Demo1234!	Usuario	100000 centavos
bruno	Demo1234!	Usuario	100000 centavos
admin	Demo1234!	Administrador	100000 centavos

El registro también funciona en ambas versiones y crea usuarios comunes con saldo cero. Las cuentas semilla permiten reproducir los ejercicios desde el mismo estado inicial.

Procedimiento antes de cada ejercicio

1. En la guía, restablecer los datos de ambas versiones y confirmar la acción. Comprobar que ambas se restablecieron correctamente.
2. Ingresar otra vez en la billetera que se observará. El reinicio cierra las sesiones anteriores y limpia los contadores de intentos.
3. Preparar las cuentas y los movimientos indicados en el ejercicio. Aplicar los mismos datos a cada versión.
4. Ejecutar el procedimiento manual o el caso automático. Revisar el estado antes y después, además de la respuesta HTTP.
5. Conservar el resultado para explicarlo. Restablecer antes de pasar a otro caso; las evidencias de la guía se mantienen.

Qué cambia en cada versión

Caso	Versión vulnerable	Versión corregida
Acceso administrativo	Basta con estar autenticado.	Se exige rol administrador.
CSRF	Acepta la mutación sin token.	Token de sesión y validación de origen.
XSS	Interpreta el concepto como HTML.	Lo muestra como texto y añade CSP.
SQLi	Concatena el término de búsqueda.	Usa parámetros y filtro de propietario.
Autenticación	Enumera cuentas y no limita intentos.	Error uniforme y límites temporales.
SSRF	Permite ambos servicios ficticios.	Sólo admite el proveedor fijo.

Para CSRF, usar localhost de forma consistente y abrir el atacante en el mismo perfil donde está iniciada la sesión víctima. Para XSS, la persona que recibe el movimiento debe abrir Actividad. Si hay un reinicio parcial, repetirlo hasta que ambas versiones queden listas.

Control de acceso administrativo

Función legítima y falla implementada

Un administrador puede listar usuarios y ajustar saldos. Un usuario común puede operar su billetera, pero no otorgarse dinero mediante una función administrativa. La falla está en require_user: el modo vulnerable comprueba la sesión y omite el rol cuando una ruta solicita admin=True.

```
# Fragmento real de backend/app.py
if admin and not teaching and user['role'] != 'admin':
    abort(403)
```

teaching es verdadero únicamente en la versión vulnerable. Por eso, las rutas /api/admin/users y /api/admin/adjustments dejan pasar a Ana en ese modo. Aunque el frontend oculte el panel, una petición directa llega al servidor.

Cómo reproducir la explotación local

1. Restablecer ambas versiones e ingresar como ana en la versión que se está probando.
2. Abrir la consola de las herramientas del navegador en esa billetera. Ejecutar el siguiente bloque para obtener el identificador real y un token válido, y solicitar un ajuste de 5000 centavos.
3. Revisar la respuesta y volver al resumen del saldo. Repetir en la otra versión desde su estado inicial. El token válido permite evaluar el control de rol sin que intervenga primero la defensa CSRF.

```
const me = await (await fetch("/api/me")).json();
const csrf = await (await fetch("/api/csrf-token")).json();
const response = await fetch("/api/admin/adjustments", {
  method: "POST",
  headers: {"Content-Type": "application/json",
            "X-CSRF-Token": csrf.token},
  body: JSON.stringify({user_id: me.user.id, delta_cents: 5000,
                        reason: "Demostración control de acceso"})
});
console.log(response.status, await response.json());
```

Corrección y evidencia observada

La versión corregida exige el rol admin en el servidor antes de listar usuarios o modificar saldos. Ana obtiene 403 y mantiene 100000 centavos. En la vulnerable obtiene 200 y pasa a 105000 centavos. El caso automático también verifica un ajuste legítimo de administrador, que responde 200 en ambas; aplica 100 centavos y los revierte para no alterar la comparación.

Cómo explicarlo

“Comprobamos una escalada de privilegios por función. Ana ya estaba autenticada, pero no tenía autorización administrativa. La solución consiste en verificar el rol en el servidor en cada operación protegida.”

CSRF en transferencias

Función legítima y falla implementada

Una transferencia autorizada nace de una acción de la persona autenticada. En CSRF, una página de otro origen induce al navegador de la víctima a enviar la operación con su sesión. El servidor vulnerable acepta el formulario sin exigir una prueba ligada a esa sesión de que la solicitud se originó en la aplicación.

En el modo vulnerable se desactiva WTF_CSRF_ENABLED y no se exige la validación de Origin. La página local del puerto 5003 envía un formulario real a /api/transfers con estos campos, sin token CSRF:

```
recipient=bruno
amount_cents=2500
concept=Demostración CSRF
```

Cómo reproducir la explotación local

1. Restablecer ambas versiones. Ingresar como ana en http://localhost:5000 y comprobar el saldo de $1.000,00.
2. En otra pestaña del mismo perfil de navegador, abrir http://localhost:5003/?variant=vulnerable. Esa página envía el formulario al backend vulnerable.
3. Volver al resumen y a Actividad. Comprobar el débito de $25,00 y el concepto Demostración CSRF.
4. Restablecer, ingresar como ana en http://localhost:5001 y abrir http://localhost:5003/?variant=secure en el mismo perfil. Comprobar que no apareció el movimiento.

Corrección y evidencia observada

La versión corregida usa CSRFProtect de Flask-WTF. Cada mutación debe incluir el token obtenido para la sesión, mediante X-CSRF-Token o el campo csrf_token. Además, comprueba que Origin coincida con el origen de la billetera cuando ese encabezado está presente.

Comprobación	Vulnerable	Corregida
Respuesta del formulario	200	400 por token ausente
Saldo de Ana en centavos	100000 → 97500	100000 → 100000
Cantidad de movimientos	0 → 1	0 → 0

En esta corrida, Flask-WTF rechaza primero la ausencia de token, por eso se observó 400. La comprobación de origen es otra capa. Una transferencia normal con token válido sigue funcionando.

Cómo explicarlo

“El navegador puede adjuntar una sesión aunque el formulario venga de otra página. Los puertos 5000 y 5003 son orígenes distintos, pero pertenecen al mismo sitio localhost; SameSite=Lax por sí solo no evita esta demostración. El token y la validación de origen protegen la operación.”

Si se usa incógnito para el atacante o se mezcla localhost con 127.0.0.1, la sesión puede no acompañar la petición y la demostración deja de evaluar la precondición prevista. CORS no sustituye la protección CSRF.

XSS persistente en conceptos

Función legítima y falla implementada

La billetera guarda el concepto de una transferencia y luego lo muestra a quien envió o recibió el movimiento. La falla aparece cuando la interfaz vulnerable interpreta ese texto como HTML. El contenido queda almacenado y se ejecuta al abrir la actividad del receptor, por eso es XSS persistente.

```
// Fragmentos de MovementList en WalletApp.tsx
// Vulnerable
<span dangerouslySetInnerHTML={{
  __html: movement.concept || "Sin concepto"
}} />
// Corregida dentro de la expresión JSX
movement.concept || "Sin concepto"
```

Cómo reproducir la explotación local

1. Restablecer e ingresar como ana. Enviar $1,00 a bruno y utilizar como concepto el siguiente marcador inocuo. Copiarlo completo; el campo permite hasta 200 caracteres.
2. Cerrar la sesión de Ana, ingresar como bruno en la misma variante y abrir Actividad. También se pueden usar perfiles separados para emisor y receptor.
3. Observar el marcador XSS ejecutado en la vulnerable. Repetir en la corregida y comprobar que se muestra el concepto como texto, sin crear el marcador.

```
<img src=x onerror="document.body.insertAdjacentHTML('afterbegin','<div data-testid=&quot;xss-
marker&quot; style=&quot;color:red;font-weight:bold&quot;>XSS ejecutado</div>');this.remove()">
```

El recurso de imagen inexistente dispara onerror. Ese manejador añade un elemento visible al documento y elimina la imagen. No extrae información ni envía datos a un servicio externo. El mismo valor se encuentra en docs/GUIA.md para copiarlo desde texto plano.

Corrección y evidencia observada

Se reemplazó el uso del contenido como HTML por la interpolación de texto de React. La CSP de la versión corregida limita el contenido al propio origen y evita manejadores inline como defensa adicional. La defensa principal es no interpretar el concepto como código.

La transferencia responde 200 en ambas versiones. Bruno pasa de 100000 a 100100 centavos y conserva el concepto en ambas bases. El contraste comprobado en Chromium es la presencia del marcador en la vulnerable y su ausencia en la corregida; rechazar la transferencia no es el objetivo de esta corrección.

Cómo explicarlo

“Almacenar un texto no lo vuelve confiable. El riesgo aparece al renderizarlo. Conservamos la transferencia y su concepto, pero evitamos que el navegador lo interprete como HTML ejecutable.”

Las cookies son HttpOnly en ambas variantes. Esta prueba demuestra ejecución mediante un marcador visual; no demuestra robo de cookies ni toma de una cuenta.

Inyección SQL en actividad

Función legítima y falla implementada

La búsqueda debe filtrar por concepto únicamente los movimientos propios. La versión vulnerable agrega el término escrito por el usuario dentro del texto SQL. Así, una entrada puede cerrar la cadena y modificar la condición que restringe las filas por propietario.

```
# Esquema abreviado de la consulta vulnerable
f"WHERE (t.sender_id={uid} OR t.recipient_id={uid}) "
f"AND t.concept LIKE '%{q}%' ORDER BY t.id DESC"
```

El fragmento abrevia la selección de columnas y llama uid al identificador de sesión. La consulta completa está en movements, en backend/app.py. La entrada del usuario se interpola únicamente en este punto didáctico; las demás operaciones de negocio usan parámetros.

Cómo reproducir la explotación local

1. Restablecer ambas versiones. En la primera, ingresar como ana y transferir $1,00 a bruno con concepto Compra propia.
2. Cerrar sesión, ingresar como admin y transferir $1,00 a bruno con concepto Movimiento reservado. Ese movimiento no pertenece a Ana.
3. Volver a ingresar como ana. Sin búsqueda, comprobar que sólo ve su propio movimiento. En Actividad, buscar el siguiente texto.
4. Repetir toda la preparación en la otra versión. Comparar las filas devueltas, no solamente el estado HTTP.

```
' OR 1=1 --
```

La comilla cierra el valor buscado, OR 1=1 vuelve verdadera la condición y -- comenta el resto de la sentencia. En este escenario de lectura, la consulta vulnerable expone también el movimiento entre Admin y Bruno.

Corrección y evidencia observada

```
# Fragmento de la versión corregida
rows = db.execute(
    select + 'WHERE (t.sender_id=? OR t.recipient_id=?) '
             'AND t.concept LIKE ? ORDER BY t.id DESC',
    (user['id'], user['id'], '%' + q + '%')
).fetchall()
```

Los signos ? reservan posiciones para valores; SQLite recibe el identificador y la búsqueda por separado de la estructura SQL. La restricción por propietario permanece fija. No se resuelve escapando a mano comillas ni confiando sólo en una validación del frontend.

Ambas respuestas observadas fueron 200. La vulnerable devolvió Compra propia y Movimiento reservado. La corregida devolvió una lista vacía para ese texto literal y no expuso el movimiento ajeno. Una búsqueda legítima sigue pudiendo recuperar el movimiento propio.

Cómo explicarlo

“En la vulnerable, el buscador permite que un dato cambie la lógica de la consulta. En la corregida, el motor recibe ese mismo contenido como un valor literal y mantiene la condición de propiedad.”

Enumeración e intentos de autenticación

Función legítima y falla implementada

La variante vulnerable distingue usuarios existentes e inexistentes mediante los errores del login y no limita los intentos. Esto facilita enumerar cuentas y continuar probando credenciales.

```
# Respuesta de error en el modo vulnerable
error = 'Contraseña incorrecta' if user else 'El usuario no existe'
# Respuesta de error en el modo corregido
error = 'Credenciales inválidas'
```

Ambas versiones almacenan hashes. El ejercicio estudia la información del error y la frecuencia de los intentos.

Cómo reproducir la explotación local

1. Restablecer ambas versiones y abrir el formulario de login de la vulnerable.
2. Probar ana con la contraseña incorrecta Incorrecta123! y luego noexiste con esa misma contraseña. Comparar los mensajes.
3. Realizar hasta seis intentos incorrectos adicionales sobre ana y observar si aparece una limitación. No es necesario intentar adivinar la contraseña demo.
4. Repetir desde el estado inicial en la corregida. Revisar en las herramientas de red los estados 401 y 429; los intentos previos consumen el límite.

Corrección aplicada

El error se unifica en Credenciales inválidas. Para usuarios inexistentes también se comprueba un hash señuelo; se evita omitir ese trabajo, aunque no se demuestra igualdad temporal perfecta entre todas las rutas.

La corregida admite hasta cinco intentos por cuenta y veinte por IP en 60 segundos. Después responde 429 y Retry-After: 60. Se contabilizan los intentos admitidos por el control, incluidos los exitosos. El reinicio limpia estos contadores.

Evidencia observada

Comprobación	Vulnerable	Corregida
Mensajes para ana y noexiste	Distintos	Credenciales inválidas en ambos
Seis intentos del bloque registrado	401 401 401 401 401 401	401 401 401 401 429 429
Límite observado	No	Sí

La primera comparación ya había consumido un intento sobre Ana. Por eso, en el bloque posterior de seis solicitudes el primer 429 aparece en la quinta posición, sin contradecir el límite configurado.

Cómo explicarlo

“La vulnerable permite conocer si existe una cuenta y seguir probando sin límite. En la corregida reducimos la información de error y limitamos la frecuencia. El caso utiliza intentos acotados y no demuestra que se haya descifrado una contraseña.”

SSRF en el cotizador

Función legítima y falla implementada

El cotizador solicita una cotización a un proveedor simulado desde el backend. SSRF aparece si el usuario logra elegir otro destino al que el servidor tiene acceso y utiliza esa función para consultar un servicio interno.

La versión vulnerable acepta destinos de dos servicios ficticios: el proveedor y private-ledger. Esto permite demostrar el cambio de destino, manteniendo el transporte confinado al laboratorio. La corregida acepta sólo la URL fija del proveedor y obtiene el destino real desde la configuración del servidor.

Cómo reproducir la explotación local

1. Restablecer ambas versiones, ingresar como ana y abrir Cotizador en la vulnerable.
2. Importar primero la URL del proveedor permitido que figura debajo. Comprobar que devuelve una cotización USD.
3. Sustituirla por la URL del servicio interno ficticio. Observar el texto DATO INTERNO FICTICIO.
4. Repetir en la corregida. La cotización legítima debe funcionar y el destino interno debe rechazarse. Los nombres se resuelven dentro de Docker; introducirlos en Cotizador, no en la barra del navegador.

```
Proveedor permitido
http://quote-provider:8080/quote/USD

Servicio interno de demostración
http://private-ledger:8080/demo
```

Corrección aplicada

```
# Fragmento del modo corregido en backend/app.py
destination = app.config['PROVIDER_URL']
# PROVIDER es la URL canónica del proveedor ficticio
if supplied != PROVIDER:
    return jsonify(error='Proveedor no permitido'), 400
# El transporte no sigue redirecciones
client.get(destination, timeout=(2, 2),
           allow_redirects=False, stream=True)
```

Además de fijar el destino, el cliente desactiva el uso de proxies del entorno, exige HTTP 200 y application/json, limita el cuerpo a 8192 bytes y aplica límites de conexión, lectura y un plazo durante el consumo del cuerpo. La corregida valida la moneda USD y una tasa entera positiva dentro del rango permitido. No basta con comprobar que la URL contenga el nombre del proveedor.

Evidencia observada y explicación

El proveedor legítimo respondió 200 en ambas. En la vulnerable, el destino interno respondió 200 y el contador de accesos pasó de 0 a 1. En la corregida se obtuvo 400 y el contador permaneció en 1: no hubo una segunda consulta. La respuesta tampoco expuso el marcador interno.

“La solicitud la hace el servidor. Corregimos la posibilidad de elegir un servicio distinto usando un destino fijo y validando la respuesta. El contenido interno es ficticio y el laboratorio no ofrece un proxy hacia Internet o la máquina anfitriona.”

Automatización y resultados comprobados

Qué hace la guía

La guía ofrece los mismos casos de forma manual y automática. El botón individual ejecuta las dos versiones contra sus instancias reales. El navegador automático tiene sesiones propias; no toma la sesión humana. Las pantallas autenticadas consultan saldo y movimientos cada dos segundos mientras están visibles, de modo que pueden mostrar los efectos persistidos sin sobrescribir formularios.

Para CSRF se usa el mismo contexto de Chromium para víctima y atacante; para XSS se separan emisor y receptor. Los casos también envían peticiones autenticadas a las API reales cuando necesitan demostrar una función administrativa oculta u observar respuestas. No se agrega un bypass a la versión corregida.

El lote completo realiza doce verificaciones secuenciales y restablece ambas billeteras antes de cada caso. Eso invalida las sesiones humanas al avanzar. Para explicar cambios en vivo conviene restablecer, ingresar como observador y ejecutar un solo caso; el lote resulta útil para revisar el informe completo.

Resultado conservado como evidencia

Corrida 000c0cb404874efcbacac65a876213c1, del 8 de noviembre de 2026: estado completed, con seis attack_reproduced en vulnerable y seis blocked en secure. Estos resultados proceden de evidencias/ejecucion-final/result.json; no son una nueva ejecución realizada para redactar el documento.

Caso	Vulnerable	Corregida
Acceso administrativo	200 y saldo +5000	403 y saldo sin cambios
CSRF	200 y transferencia de 2500	400 y ningún movimiento
XSS persistente	200 y marcador visible	200 y concepto como texto
SQLi	200 y movimiento ajeno	200 sin movimiento ajeno
Autenticación	Mensajes distintos y sin 429	Mensajes uniformes y 429
SSRF	200 y una consulta interna	400 y ninguna consulta adicional

Cómo leer el resultado sin confundirlo

attack_reproduced indica que se observó el efecto vulnerable esperado. blocked representa la defensa verificada para ese escenario, incluso cuando el HTTP es 200. unexpected señala una diferencia respecto de lo esperado. Un timeout, una excepción o la falta de precondiciones produce un resultado inconcluso; una cancelación tampoco acredita una defensa.

Se conservan solicitudes y respuestas resumidas sin contraseñas, cookies ni tokens, valores antes y después y capturas. La guía retiene las últimas diez ejecuciones en almacenamiento independiente del reset; la exportación final permanece en la carpeta de evidencias. Cancelar no revierte operaciones que ya hayan ocurrido.

La validación registrada del proyecto incluye 21 pruebas de backend y guía, compilación y comprobación de TypeScript, comprobaciones en Chromium y revisión de la interfaz a 1440 px y 390 px. Son verificaciones de alcance definido, no una auditoría integral.

Guion y preguntas para la exposición

Recorrido sugerido

Este orden puede adaptarse al tiempo que asigne la cátedra. Antes de exponer, ensayar los pasos manuales en el equipo que se utilizará y disponer de las evidencias exportadas para explicar los resultados aunque falle una demostración en vivo.

1. Presentar Umbral como una billetera ficticia. Mostrar las versiones vulnerable y corregida, la guía y las cuentas de demostración.
2. Explicar que el negocio, los datos iniciales y el diseño son equivalentes. Mostrar el reset y advertir que invalida las sesiones.
3. Demostrar control de acceso con Ana y explicar por qué una sesión válida no otorga permisos administrativos.
4. Mostrar CSRF con dos pestañas del mismo perfil y XSS desde la actividad de Bruno. Relacionar cada efecto con su precondición.
5. Mostrar SQLi con un movimiento propio y otro ajeno. Comparar las filas devueltas.
6. Comparar errores y límites del login. Mostrar el cotizador legítimo y el servicio interno ficticio.
7. Abrir el reporte de las doce verificaciones. Elegir una evidencia y explicar solicitud, estado antes y después y corrección correspondiente.
8. Cerrar indicando el alcance de las seis correcciones y las limitaciones que siguen existiendo en un laboratorio local.

Por qué conservamos una versión vulnerable

Permite aislar la causa de cada falla y comparar el efecto de incorporar un control concreto. No se eliminaron las validaciones básicas de dinero ni el hash de contraseñas para provocar errores ajenos a los seis casos. El modo vulnerable queda fijado al arrancar y contenido en un entorno de enseñanza.

Qué diferencia hay entre autenticación y autorización

La autenticación comprueba quién es la persona. La autorización decide qué operaciones puede realizar. Ana supera el login, pero no tiene permiso para ajustar saldos. El caso de roles estudia esa segunda decisión; los mensajes y límites del login estudian la primera.

Qué diferencia hay entre CSRF y XSS

CSRF usa la sesión que el navegador adjunta a una solicitud iniciada desde otro origen. XSS consigue ejecutar contenido dentro de la página de la aplicación. El token CSRF no corrige un concepto interpretado como HTML. React como texto y CSP se ocupan del segundo problema.

Por qué la defensa puede responder con 200

En XSS, la transferencia es legítima y debe guardarse, pero el concepto debe mostrarse sin ejecutarlo. En SQLi, una búsqueda válida puede no encontrar coincidencias. Por eso se revisa el efecto y los datos, además del código HTTP.

Qué aporta la automatización

Repite la preparación y las comprobaciones con datos definidos, conserva evidencia y permite detectar cambios en el comportamiento. La explicación manual muestra por qué ocurre cada caso. Un reporte completo debe acompañarse de su evidencia y no reemplaza revisar el código.

Límites pendientes y archivos de referencia

Alcance y puntos que deben explicarse

La versión corregida resuelve las seis fallas diseñadas y verificadas. Usa HTTP local, por lo que las cookies no llevan Secure ni existe cifrado de transporte. El despliegue, el almacenamiento de sesiones en archivos y los límites locales de intentos responden al laboratorio; no constituyen una arquitectura bancaria de producción.

La restricción SSRF depende de destinos ficticios y del DNS interno confiable. No se ha demostrado una solución general contra toda manipulación de red o DNS. Las revisiones de dependencias de Python y frontend no abarcan por sí solas el sistema operativo de las imágenes ni todo Chromium.

Pendiente técnico conocido: un fallo de escritura al guardar el estado de una corrida puede dejar ocupado el ejecutor. Se identificó en la revisión de lab/runner.py y todavía requiere una corrección que libere siempre el bloqueo. No afecta la evidencia de la corrida exitosa conservada, pero debe resolverse antes de afirmar que el ejecutor recupera todos los errores de almacenamiento.

El reinicio de las dos bases no es una única transacción distribuida. Cada billetera restaura sus datos de forma transaccional; si falla una, la guía informa el resultado parcial y bloquea nuevas pruebas hasta completar ambos reinicios. También queda por ensayar el recorrido manual completo en el equipo de exposición y comprobar allí el arranque sin Internet con las imágenes ya disponibles.

Dónde revisar lo explicado

Las rutas siguientes son relativas a la carpeta tpi-billetera del proyecto. Los fragmentos del documento simplifican la presentación; estos archivos contienen la implementación y la evidencia completa.

Archivo o carpeta	Contenido que permite verificar
backend/app.py	Sesiones, roles, CSRF, SQL, login, cotizador y transacciones.
frontend/src/wallet/WalletApp.tsx	Renderizado del concepto y pantallas de la billetera.
lab/api_cases.py y tests_browser/scenarios.py	Preparación y comprobación de las seis demostraciones.
lab/runner.py	Ejecuciones, serialización, cancelación y reinicio.
compose.yaml y fixtures/	Red, servicios simulados y contención local.
evidencias/ejecucion-final/result.json	Doce resultados y valores exactos antes y después.
evidencias/ejecucion-final/	Capturas y resultados por caso y versión.
docs/GUIA.md y docs/INFORME.md	Pasos copiables y explicación técnica ampliada.

Criterio para justificar una corrección

Para cada falla, presentar la precondición, la petición o entrada concreta, el efecto observado y el cambio de código. Mostrar después que la operación legítima sigue funcionando y que el mismo escenario ya no produce el efecto indebido. Si falta una de esas comprobaciones, describir lo que sí se verificó y dejar lo demás como pendiente.